

全栈智能系统管理中心构建指南

本文档融合了网络管理、硬件监控、桌面自动化以及AI诊断的综合需求，为您量身定制了 **Windows 桌面版系统综合管理中心** 的落地开发指南。本方案采用前后端分离架构，结合轻量级、高性能的 Python (FastAPI) 后端与现代化的 Vue 3 前端，能够稳定承载高频次的数据抓取与外部 API (DeepSeek/Feishu) 调用。

一、架构总览 (Architecture)

技术栈选型

- 后端中枢:** Python 3.10+ & FastAPI (极低的开发成本，强大的系统级库 `psutil` 配合 `WMI` 深度控制 Windows)。
- 前端渲染:** Vue 3 + Vite + Tailwind CSS (构建暗黑科技风仪表盘)。
- 内网穿透与安全:** Cloudflare Tunnel (Zero Trust) 暴露内网服务，后端强制集成 `PyOTP` 进行二次验证。
- AI 诊断大脑:** DeepSeek API 作为主力云端推理，本地可随时切换 Nvidia GPU 驱动的本地化模型。
- 消息总线:** 飞书 (Feishu) 自定义机器人，实现告警推送与指令逆向下发。

二、核心模块功能定义 (Modules)

我们对基础模块和扩展模块进行了深度融合：

- 全局网络与资产监控:** 监控本机网卡状态、局域网设备扫描；深度集成 Synology NAS 状态读取与 tv 盒子在线监控。
- VPS与代理矩阵:** 集中管理名下的 RackNerd、CloudCone 等云服务器实例，监控其存活状态及网络延迟。
- Windows 系统级调度:** 实时监控系统健康状态，读取注册表扫描桌面应用，实现服务的可视化启停。
- 自动化剧本中枢:** 支持通过 Webhook 与本地 Docker 部署的 n8n 工作流引擎联动，执行定时任务和自动化剧本。

三、 后端核心代码实现 (FastAPI)

以下代码为系统的核心框架 `main.py`，展示了安全认证、系统信息抓取、AI 联动与飞书推送的实现方式。

```

import uvicorn
from fastapi import FastAPI, Header, HTTPException, Depends
from pydantic import BaseModel
import psutil
import pyotp
import requests
import subprocess
import os

app = FastAPI(title="SysCenter API")

# 1. OTP 动态验证中间件
# 在实际应用中，此密钥应安全存储在环境变量中，此处为演示
OTP_SECRET = pyotp.random_base32()

def verify_otp(x_otp_token: str = Header(None)):
    if not x_otp_token:
        raise HTTPException(status_code=403, detail="需要 OTP 令牌")
    totp = pyotp.TOTP(OTP_SECRET)
    if not totp.verify(x_otp_token):
        raise HTTPException(status_code=403, detail="OTP 验证失败")
    return True

# 2. 系统健康与进程读取 (需 OTP 验证)
@app.get("/api/system/health", dependencies=[Depends(verify_otp)])
def get_system_health():
    return {
        "cpu_percent": psutil.cpu_percent(interval=1),
        "ram_percent": psutil.virtual_memory().percent,
        "disk_usage": psutil.disk_usage('/').percent,
        "network": psutil.net_io_counters()._asdict()
    }

# 3. 局域网服务状态不可操作说明接口
@app.get("/api/modules/info", dependencies=[Depends(verify_otp)])
def get_module_info(module_name: str):
    info_map = {
        "docker": "【说明】容器管理已被隔离。如需管理容器节点及挂载的夸克网盘应用，请移步局域网内的 Portainer 或 aaPanel 面板。",
        "core_service": "【警告】此 Windows 核心服务禁止在此面板直接关闭，以免导致系统崩溃。"
    }
    return {"module": module_name, "instruction": info_map.get(module_name, "暂无说明")}

# 4. DeepSeek AI 智能诊断集成
class LogRequest(BaseModel):
    log_content: str

@app.post("/api/ai/diagnose", dependencies=[Depends(verify_otp)])

```

```

def deepseek_diagnose(req: LogRequest):
    API_KEY = os.getenv("DEEPSEEK_API_KEY", "your_deepseek_key")
    url = "https://api.deepseek.com/v1/chat/completions"

    prompt = f"作为高级Windows与网络运维工程师，请分析以下系统异常日志并给出分步排障指导：{req.log_content}"

    payload = {
        "model": "deepseek-chat",
        "messages": [{"role": "user", "content": prompt}]
    }
    headers = {"Authorization": f"Bearer {API_KEY}", "Content-Type": "application/json"}

    try:
        response = requests.post(url, json=payload, headers=headers)
        return response.json()
    except Exception as e:
        return {"error": str(e)}

# 5. 飞书 (Feishu) 机器人告警推送
class AlertRequest(BaseModel):
    message: str

@app.post("/api/notify/feishu")
def send_feishu_alert(req: AlertRequest):
    WEBHOOK_URL = os.getenv("FEISHU_WEBHOOK", "your_feishu_webhook_url")
    payload = {
        "msg_type": "text",
        "content": {"text": f"🚨 [SysCenter 告警] {req.message}"}
    }
    requests.post(WEBHOOK_URL, json=payload)
    return {"status": "success"}

if __name__ == "__main__":
    # 生产环境中推荐配合 uvicorn 与反向代理运行
    uvicorn.run(app, host="0.0.0.0", port=8000)

```

提示： 上述代码利用 `psutil` 快速实现了底层系统探针功能。如果要实现外网与内网本机的深度控制（例如重启某些服务或启动桌面应用），可以通过 Python 的 `subprocess.Popen` 或 `os.system` 封装命令行调用。

四、前端集成与交互逻辑 (Vue 3 示例)

前端需要处理 OTP 弹窗以及对不可操作模块的说明提示。以下是一个使用 Axios 进行带验证请求的基础组件示例：

```

<template>
  <div class="dashboard">
    <div class="header">
      <h1>系统综合管理面板</h1>
      <input v-model="otpToken" placeholder="输入 6 位 OTP" />
    </div>

    <div class="card">
      <h3>系统健康</h3>
      <p>CPU 占用: {{ sysData.cpu_percent }}%</p>
      <button @click="fetchHealth">刷新状态</button>
    </div>

    <div class="card">
      <h3>Docker 容器引擎</h3>
      <!-- 不可操作模块, 点击弹出说明 -->
      <button class="info-btn" @click="showModuleInfo('docker')">
        查看模块说明 ( i )
      </button>
    </div>
  </div>
</template>

<script setup>
import { ref } from 'vue';
import axios from 'axios';

const otpToken = ref('');
const sysData = ref({});

const api = axios.create({ baseURL: 'http://localhost:8000' });

// 拦截器注入 OTP
api.interceptors.request.use(config => {
  if (otpToken.value) config.headers['x-otp-token'] = otpToken.value;
  return config;
});

const fetchHealth = async () => {
  try {
    const res = await api.get('/api/system/health');
    sysData.value = res.data;
  } catch (error) {
    alert("验证失败或网络错误");
  }
};

const showModuleInfo = async (module) => {

```

```
try {
  const res = await api.get(`/api/modules/info?module_name=${module}`);
  alert(res.data.instruction);
} catch(e) {
  alert("请先输入正确的OTP口令以访问详细说明。");
}
};
</script>
```

五、部署与运行步骤

1. **初始化环境：** 在 Windows 宿主机上安装 Python 3.10+, 并执行 `pip install fastapi uvicorn psutil pyotp requests pydantic`。
2. **生成 OTP 密钥：** 运行脚本提取 `OTP_SECRET` 并导入手机端的 Authenticator（谷歌或微软身份验证器）。
3. **启动后端：** 运行 `python main.py`。
4. **Cloudflare Tunnel 配置：** 在本机下载 `cloudflared`，通过命令行执行 `cloudflared tunnel route dns [tunnel-id] [your-domain]` 绑定域名，并将 `http://localhost:8000` 作为 Service 暴露出去。
5. **飞书告警挂载：** 在飞书群助手后台生成 Webhook 地址，将地址填入后端的环境变量 `FEISHU_WEBHOOK` 中。配合系统的定时任务，即可实现宕机预警。